Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica
Tópicos especiais em sistemas embarcados

Atividade FreeRTOS para ESP32

VALDEMAR GONÇALVES PEREIRA NETO

Atividade referente a primeira entrega avaliativa para a disciplina ELE0629 - Top. Esp. Sistemas Embarcados ministrada pelo PROF. DR. ANTONIO WALLACE ANTUNES SOARES com docente assistido JOSÉ IGO SOARES CAMILO

Natal, 30 de março de 2026

Resumo

O uso do *kernel* de tempo real FreeRTOS integrado ao microcontrolador ESP32 via ESP-IDF constitui uma plataforma robusta para o desenvolvimento de aplicações embarcadas com requisitos determinísticos, sendo amplamente adotada em sistemas industriais, médicos e de IoT. Nesse contexto, foram desenvolvidas três atividades práticas com o objetivo de explorar os fundamentos do FreeRTOS no ESP32, abordando progressivamente a inspeção de tarefas do sistema, a medição do *overhead* de troca de contexto do *kernel* e a proteção de recursos compartilhados com semáforos. Para isso, utilizou-se o módulo ESP32-WROOM-32 como plataforma de *hardware*, o ambiente *Visual Studio Code* com a extensão ESP-IDF v5.5.1, e as APIs nativas do FreeRTOS, destacando-se as funções `uxTaskGetSystemState()`, `esp_timer_get_time()`, `xSemaphoreCreateMutex()` e `xSemaphoreCreateCounting()`. A primeira atividade identificou cinco tarefas internas instanciadas automaticamente pelo ESP-IDF, incluindo as tarefas ociosas e de comunicação entre núcleos. A segunda atividade registrou 101 trocas de contexto ao longo de 1 segundo, com tempo médio de 44.459 μs e *overhead* percentual de $4,4459\%$. A terceira atividade demonstrou o correto funcionamento do padrão produtor-consumidor com sete tarefas concorrentes, garantindo exclusão mútua e sincronização por meio de três semáforos. Os resultados obtidos confirmam o caráter determinístico e eficiente do FreeRTOS, validando sua aplicabilidade em sistemas embarcados com requisitos de tempo real sobre o hardware dual-core do ESP32.

FreeRTOS. ESP32. ESP-IDF. Sistemas Embarcados.

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
ADC	*Analog-to-Digital Converter* — Conversor Analógico-Digital
API	*Application Programming Interface* — Interface de Programação de Aplicações
BLE	*Bluetooth Low Energy* — Bluetooth de Baixo Consumo
BT	*Bluetooth*
CPU	*Central Processing Unit* — Unidade Central de Processamento
DAC	*Digital-to-Analog Converter* — Conversor Digital-Analógico
DEE	Departamento de Engenharia Elétrica
ESP	*Espressif Systems Platform*
GND	*Ground* — Terra
GPIO	*General Purpose Input/Output* — Entrada/Saída de Propósito Geral
I2C	*Inter-Integrated Circuit* — Protocolo de Comunicação entre Circuitos Integrados
IDF	*IoT Development Framework*
IoT	*Internet of Things* — Internet das Coisas
IPC	*Inter-Processor Communication* — Comunicação entre Processadores
MHz	Megahertz
MISO	*Master In Slave Out*
MOSI	*Master Out Slave In*
RTOS	*Real-Time Operating System* — Sistema Operacional de Tempo Real
SCL	*Serial Clock Line* — Linha de Clock Serial
SDA	*Serial Data Line* — Linha de Dados Serial
SMP	*Symmetric Multiprocessing* — Multiprocessamento Simétrico

SoC	*System on Chip* — Sistema em Chip
SPI	*Serial Peripheral Interface* — Interface Serial para Periféricos
SRAM	*Static Random Access Memory* — Memória de Acesso Aleatório Estática
UART	*Universal Asynchronous Receiver-Transmitter* — Receptor-Transmissor Assíncrono Universal
UFRN	Universidade Federal do Rio Grande do Norte
USB	*Universal Serial Bus* — Barramento Serial Universal
VSCode	*Visual Studio Code*
Wi-Fi	*Wireless Fidelity*

Lista de tabelas

Tabela 1 – Especificações técnicas do ESP32-WROOM-32 9
Tabela 2 – Descrição de pinos do ESP32-WROOM-32 10
Tabela 3 – Tarefas do sistema identificadas na Atividade 1 31
Tabela 4 – Resultados da medição de overhead do kernel — Atividade 2 32

Lista de Imagens

Figura 1 – ESP32-WROOM-32 *Pin Layout (Top View)* 10
Figura 2 – Ambiente de desenvolvimento: VSCode com extensão ESP-IDF v5.5.1 . 13
Figura 3 – Transições válidas entre estados de tarefas 16
Figura 4 – Diagrama temporal da medição de troca de contexto 21
Figura 5 – Diagrama de eventos do sistema produtor-consumidor — Atividade 3 . 26
Figura 6 – Saída do monitor serial 30
Figura 7 – Saída do monitor serial — Atividade 2 32
Figura 8 – Saída do monitor serial — Atividade 3 33

Sumário

1 INTRODUÇÃO . . . 8

2 MATERIAIS E MÉTODOS . . . 9

2.1 Materiais . . . 9

2.1.1 Hardware . . . 9

2.1.2 Software . . . 12

2.2 Metodologia . . . 13

3 IMPLEMENTAÇÃO . . . 15

3.1 ATIVIDADE 1 — INSPEÇÃO DE TAREFAS DO SISTEMA . . . 15

3.1.1 Descrição da Atividade . . . 15

3.1.2 Fundamentação Teórica . . . 15

3.1.2.1 Tarefas no FreeRTOS . . . 15

3.1.2.2 A Função `uxTaskGetSystemState()` . . . 16

3.1.2.3 Uso de CPU por Tarefa . . . 17

3.1.2.4 Memória Heap Disponível . . . 17

3.1.3 Implementação . . . 17

3.2 ATIVIDADE 2 — MEDIÇÃO DO OVERHEAD DO KERNEL . . . 19

3.2.1 Descrição da Atividade . . . 19

3.2.2 Fundamentação Teórica . . . 19

3.2.2.1 Troca de Contexto no FreeRTOS . . . 19

3.2.2.2 A Função `esp_timer_get_time()` . . . 20

3.2.2.3 Metodologia de Medição . . . 20

3.2.3 Implementação . . . 21

3.3 ATIVIDADE 3 — PROTEÇÃO DE RECURSO COMPARTILHADO COM SEMÁFOROS . . . 23

3.3.1 Descrição da Atividade . . . 23

3.3.2 Fundamentação Teórica . . . 23

3.3.2.1 O Problema do Produtor-Consumidor . . . 23

3.3.2.2 Semáforo Mutex . . . 24

3.3.2.3 Semáforos Contadores . . . 24

3.3.2.4 Uso do Parâmetro `void *parameters` . . . 24

3.3.3 Implementação . . . 25

4 RESULTADOS . . . 30

4.1 Atividade 1 . . . 30

4.2 Atividade 2 32
4.3 Atividade 3 32

5 CONCLUSÃO 34

REFERÊNCIAS 36

1 INTRODUÇÃO

A integração entre *FreeRTOS* e o microcontrolador ESP32 constitui uma ferramenta robusta e eficaz para o desenvolvimento de soluções a problemas complexos e desafiadores em sistemas embarcados. Com capacidade de processamento *dual-core*, o ESP32 é compatível com o paradigma de programação orientada a eventos e com aplicações que demandam baixa latência e respostas determinísticas. Nesse contexto, a Espressif Systems, uma das principais desenvolvedoras de SoCs (*System on Chip*) do mundo, criou o ESP-IDF (*Espressif IoT Development Framework*), o *framework* oficial de desenvolvimento para SoCs das famílias ESP32, ESP32-S, ESP32-C, ESP32-H e ESP32-P.

Um RTOS (*Real-Time Operating System*) é um sistema operacional que garante execução previsível e determinística, essencial em sistemas embarcados com requisitos de tempo real. O *FreeRTOS*, adotado como *kernel* de tempo real no ESP-IDF, é amplamente utilizado em aplicações industriais, médicas e de IoT, sendo suas principais características:

- **Leveza e compacidade:** projetado para rodar em microcontroladores com severas restrições de memória e processamento;
- **Modelo de tarefas:** cada *thread* de execução é denominada *task*, configurável com prioridade, tamanho de pilha, nome e núcleo de execução definidos pelo desenvolvedor;
- **Determinismo:** garantia de que eventos críticos serão tratados dentro de um tempo máximo conhecido, viabilizada por um *scheduler* baseado em prioridades e por operações internas de tempo constante.

Diante desse contexto, o presente relatório descreve o desenvolvimento de três atividades práticas com FreeRTOS no ESP32, utilizando o ESP-IDF como *framework* de suporte. As atividades abordam, respectivamente: a inspeção das tarefas em execução no sistema, a medição do tempo de troca de contexto pelo *kernel*, e a implementação de um mecanismo de proteção de recurso compartilhado utilizando semáforos.

2 MATERIAIS E MÉTODOS

2.1 Materiais

2.1.1 Hardware

O hardware utilizado nas atividades foi o módulo ESP32-WROOM-32, desenvolvido pela Espressif Systems. Trata-se de um módulo Wi-Fi + Bluetooth + BLE de propósito geral, voltado para uma ampla variedade de aplicações, desde redes de sensores de baixo consumo até tarefas mais exigentes, como codificação de voz e *streaming* de áudio. A Tabela 1 apresenta as principais especificações técnicas do módulo.

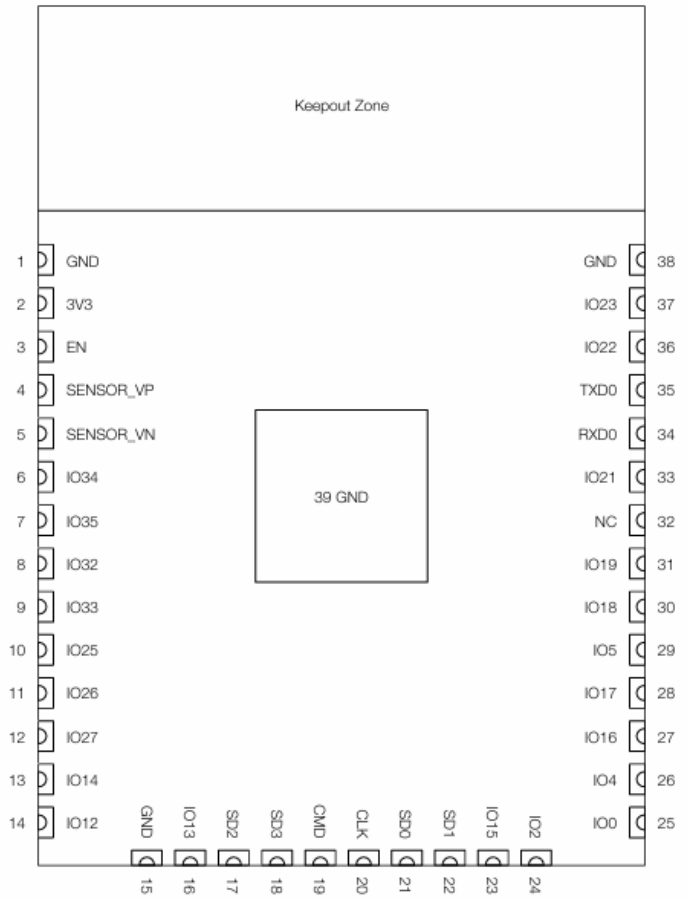
Tabela 1 – Especificações técnicas do ESP32-WROOM-32

Característica	**Especificação**
SoC	ESP32-D0WDQ6
Arquitetura	Xtensa® 32-bit LX6
Núcleos	Dual-core
Frequência de clock	80 MHz a 240 MHz
SRAM interna	520 KB
Flash externa (SPI)	4 MB
Conectividade	Wi-Fi 802.11 b/g/n, BT, BLE
Tensão de operação	3,3 V
Temperatura de operação	−40°C a +85°C
Dimensões	18 × 25,5 × 3,1 mm

Fonte: Adaptado de Espressif Systems (2023)

A Figura 1 ilustra o *layout* de pinos do módulo em vista superior.

Figura 1 – ESP32-WROOM-32 *Pin Layout (Top View)*



Fonte: Espressif Systems (2023)

O ESP32-WROOM-32 tem 38 pinos. A tabela 2 apresenta a descrição de cada pino presente no SOC utilizado.

Tabela 2 – Descrição de pinos do ESP32-WROOM-32

Nome	**N°**	**Tipo**	**Função**
GND	1	P	Terra (GND)
3V3	2	P	Alimentação — 3,3 V
EN	3	I	Habilitação do módulo. Nível alto ativo
SENSOR_VP	4	I	GPIO36 — entrada ADC1_CH0 / sensor de tensão
SENSOR_VN	5	I	GPIO39 — entrada ADC1_CH3 / sensor de tensão
IO34	6	I	GPIO34 — entrada digital / ADC1_CH6

(continua na próxima página)

(continua)

Nome	**N°**	**Tipo**	**Função**
IO35	7	I	GPIO35 — entrada digital / ADC1_CH7
IO32	8	I/O	GPIO32 — entrada/saída digital / ADC1_CH4 / Touch9
IO33	9	I/O	GPIO33 — entrada/saída digital / ADC1_CH5 / Touch8
IO25	10	I/O	GPIO25 — entrada/saída digital / ADC2_CH8 / DAC1
IO26	11	I/O	GPIO26 — entrada/saída digital / ADC2_CH9 / DAC2
IO27	12	I/O	GPIO27 — entrada/saída digital / ADC2_CH7 / Touch7
IO14	13	I/O	GPIO14 — entrada/saída digital / ADC2_CH6 / Touch6 / MTMS
IO12	14	I/O	GPIO12 — entrada/saída digital / ADC2_CH5 / Touch5 / MTDI
GND	15	P	Terra (GND)
IO13	16	I/O	GPIO13 — entrada/saída digital / ADC2_CH4 / Touch4 / MTCK
SD2 (*)	17	I/O	GPIO9 — SPI Flash SHD/SD2
SD3 (*)	18	I/O	GPIO10 — SPI Flash SWP/SD3
CMD (*)	19	I/O	GPIO11 — SPI Flash SCS/CMD
CLK (*)	20	I/O	GPIO6 — SPI Flash SCK/CLK
SD0 (*)	21	I/O	GPIO7 — SPI Flash SDO/SD0
SD1 (*)	22	I/O	GPIO8 — SPI Flash SDI/SD1
IO15	23	I/O	GPIO15 — entrada/saída digital / ADC2_CH3 / Touch3 / MTDO
IO2	24	I/O	GPIO2 — entrada/saída digital / ADC2_CH2 / Touch2
IO0	25	I/O	GPIO0 — entrada/saída digital / ADC2_CH1 / Touch1
IO4	26	I/O	GPIO4 — entrada/saída digital / ADC2_CH0 / Touch0
IO16	27	I/O	GPIO16 — entrada/saída digital / UART2 RX
IO17	28	I/O	GPIO17 — entrada/saída digital / UART2 TX

(continua na próxima página)

(continua)

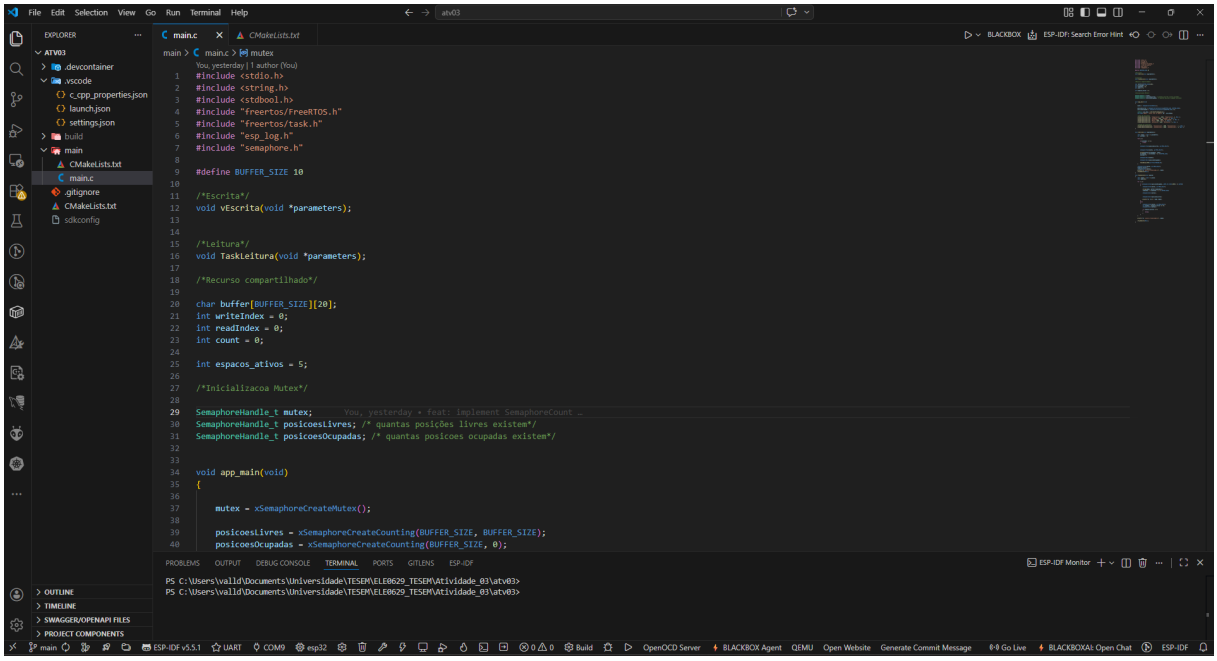
Nome	**N°**	**Tipo**	**Função**
IO5	29	I/O	GPIO5 — entrada/saída digital / VSPI CS
IO18	30	I/O	GPIO18 — entrada/saída digital / VSPI CLK
IO19	31	I/O	GPIO19 — entrada/saída digital / VSPI MISO
NC	32	—	Não conectado
IO21	33	I/O	GPIO21 — entrada/saída digital / I2C SDA
RXD0	34	I/O	GPIO3 — UART0 RX
TXD0	35	I/O	GPIO1 — UART0 TX
IO22	36	I/O	GPIO22 — entrada/saída digital / I2C SCL
IO23	37	I/O	GPIO23 — entrada/saída digital / VSPI MOSI
GND	38	P	Terra (GND)
GND	39	P	Terra (GND) — pino de fixação

(*) Pinos GPIO6 a GPIO11 estão conectados internamente à memória SPI Flash do módulo e **não devem ser utilizados** para outras funções.
Tipos: P = alimentação, I = entrada, O = saída, I/O = bidirecional.
Fonte: Adapatado de Espressif Systems (2023)

2.1.2 Software

O ambiente de desenvolvimento utilizado foi o *Visual Studio Code* (VSCode), editor de código-fonte amplamente adotado na comunidade de sistemas embarcados por sua extensibilidade e suporte a diversas linguagens e ferramentas. A integração com o ESP32 foi realizada por meio da extensão oficial da Espressif para o VSCode, que incorpora o ESP-IDF na versão 5.5.1 diretamente ao ambiente de desenvolvimento, conforme ilustrado na Figura 2.

Figura 2 – Ambiente de desenvolvimento: VSCode com extensão ESP-IDF v5.5.1



Fonte: Elaborado pelo autor

O ESP-IDF fornece um conjunto completo de APIs, drivers de periféricos, sistema de *build* baseado em CMake e integração nativa com o *kernel* FreeRTOS, sendo a plataforma recomendada para desenvolvimento profissional e acadêmico com o ESP32.

As documentações de referência utilizadas ao longo do desenvolvimento foram:

- **ESP-IDF Programming Guide v5.5.1** (Espressif Systems, 2025), documentação oficial da Espressif, contendo referências de API, guias de configuração e exemplos de uso do *framework*;
- **FreeRTOS Reference Manual** (FreeRTOS, 2024a), documentação oficial do *kernel* FreeRTOS, com descrição detalhada das primitivas de sincronização, gerenciamento de tarefas e temporização.

2.2 Metodologia

O desenvolvimento das atividades seguiu uma abordagem incremental, em que cada atividade introduz um novo conceito do FreeRTOS, construindo sobre os conhecimentos adquiridos na atividade anterior. O código-fonte desenvolvido nas três atividades está disponível publicamente no repositório do projeto, hospedado na plataforma GitHub (NETO, 2026) [1], e organizado seguindo a estrutura padrão de projetos ESP-IDF, com um

[1] Disponível em: <https://github.com/Valdemar-Neto/ELE0629_TESEM> . Acesso em: 29 mar. 2026.

diretório independente para cada atividade. A comunicação dos resultados foi realizada por meio do monitor serial, acessado via interface do próprio VSCode com a extensão ESP-IDF.

A seguir, apresenta-se uma visão geral da abordagem adotada em cada atividade, cujos detalhes de implementação são discutidos nas seções subsequentes.

- **Atividade 1: Inspeção de tarefas:** criação de uma tarefa periódica responsável por listar, a cada 5 segundos, todas as tarefas em execução no sistema, incluindo nome, estado, prioridade, tamanho de pilha e núcleo de execução;
- **Atividade 2: Medição do tempo de troca de contexto:** criação de duas tarefas e medição do tempo exclusivo do *kernel* durante a troca de contexto entre elas, utilizando a função `esp_timer_get_time()` da biblioteca `esp_timer.h`. As medições foram realizadas N vezes e o valor médio foi calculado para maior precisão;
- **Atividade 3: Proteção de recurso compartilhado com semáforos:** implementação de um sistema produtor-consumidor com cinco tarefas produtoras e duas tarefas leitoras, compartilhando um *buffer* global de dez posições. O acesso concorrente ao recurso foi controlado por semáforos contadores e Mutex, prevenindo condições de corrida e garantindo a integridade dos dados.

3 IMPLEMENTAÇÃO

3.1 ATIVIDADE 1 — INSPEÇÃO DE TAREFAS DO SISTEMA

3.1.1 Descrição da Atividade

A primeira atividade consistiu em identificar quantas e quais tarefas estão em execução durante o funcionamento do ESP32, criando uma tarefa de monitoramento que, a cada 5 segundos, reporta no monitor serial as seguintes informações de cada tarefa ativa: nome, estado, prioridade, tamanho de pilha e núcleo de execução.

3.1.2 Fundamentação Teórica

3.1.2.1 Tarefas no FreeRTOS

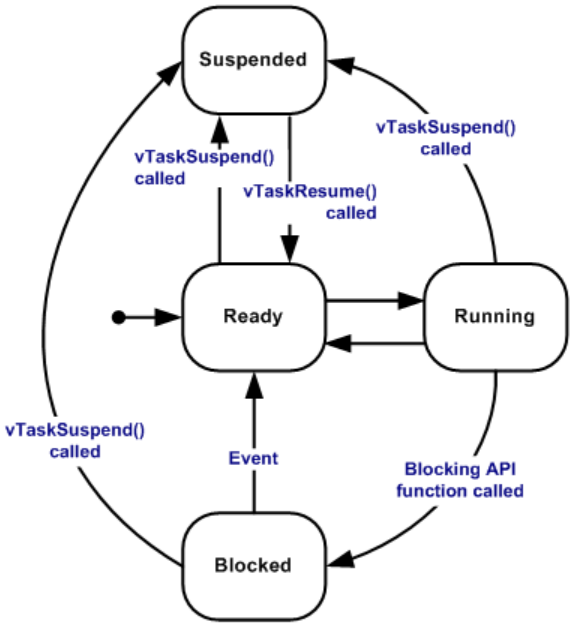
No FreeRTOS, cada unidade de execução independente é denominada *task*. Ao inicializar, o ESP-IDF cria automaticamente um conjunto de tarefas internas responsáveis pelo gerenciamento do sistema, como o escalonador, as tarefas ociosas (*idle tasks*) de cada núcleo e o *timer daemon*.

Cada tarefa no FreeRTOS pode se encontrar em um dos seguintes estados:

- **Running:** a tarefa está atualmente em execução no processador;
- **Ready:** a tarefa está pronta para executar, aguardando o escalonador ceder o processador;
- **Blocked:** a tarefa está aguardando um evento externo, como o vencimento de um *delay* ou a disponibilidade de um recurso;
- **Suspended:** a tarefa foi explicitamente suspensa e não compete pelo processador até ser retomada.

A forma que cada tarefa é acionada está descrita na Figura 3.

Figura 3 – Transições válidas entre estados de tarefas



Fonte: FreeRTOS (2024a)

3.1.2.2 A Função `uxTaskGetSystemState()`

A principal função utilizada nesta atividade foi a `uxTaskGetSystemState()`, disponível na API de utilitários de tarefas do FreeRTOS (FreeRTOS, 2024c). Essa função popula um array de estruturas `TaskStatus_t`, uma para cada tarefa sob controle do RTOS, contendo informações como o *handle* da tarefa, nome, prioridade, estado atual e o tempo total de CPU consumido pela tarefa. Sua assinatura é:

```
UBaseType_t uxTaskGetSystemState(
    TaskStatus_t * const pxTaskStatusArray,
    const UBaseType_t    uxArraySize,
    configRUN_TIME_COUNTER_TYPE * const pulTotalRunTime
);
```

Listing 3.1 – Assinatura da função uxTaskGetSystemState()

onde os parâmetros são:

- `pxTaskStatusArray`: ponteiro para o array de estruturas `TaskStatus_t` a ser preenchido;
- `uxArraySize`: tamanho do array, em número de estruturas — obtido previamente com `uxTaskGetNumberOfTasks()`;
- `pulTotalRunTime`: ponteiro para a variável que receberá o tempo total de execução do sistema desde o boot, utilizado para calcular o percentual de uso de CPU por tarefa.

É importante destacar que, para que essa função esteja disponível, é necessário que a macro `configUSE_TRACE_FACILITY` esteja definida como `1` no arquivo `FreeRTOSConfig.h` (FreeRTOS, 2024a). No ESP-IDF, essa configuração pode ser habilitada via `idf.py menuconfig`.

3.1.2.3 Uso de CPU por Tarefa

Além das informações básicas exigidas pela atividade, o código foi incrementado com o cálculo do percentual de uso de CPU por tarefa. O campo `ulRunTimeCounter` da estrutura `TaskStatus_t` armazena o tempo acumulado em que cada tarefa esteve no estado *Running* (FreeRTOS, 2024a). Dividindo esse valor pelo tempo total de execução do sistema, também fornecido por `uxTaskGetSystemState()`, obtém-se o percentual de uso de CPU:

$$\text{Uso de CPU } (\%) = \frac{\texttt{ulRunTimeCounter}}{\texttt{ulTotalRunTime}/100} \tag{3.1}$$

Quando o valor calculado é zero, indica que a tarefa consumiu menos de 1% do tempo total de CPU desde o boot do sistema.

3.1.2.4 Memória Heap Disponível

O código também monitora a memória *heap* livre por meio da função xPortGetFreeHeapSize(), que retorna, em bytes, a quantidade de memória disponível no *heap* do FreeRTOS no momento da chamada (FreeRTOS, 2024d). Esse dado é relevante para identificar possíveis vazamentos de memória em tempo de execução.

3.1.3 Implementação

A solução foi estruturada em uma única tarefa, `vMonitorTask`, criada na função `app_main()` com prioridade 5 e pilha de 10.000 bytes, valor dimensionado para acomodar o buffer de saída e o array de estruturas `TaskStatus_t` alocado dinamicamente via `pvPortMalloc()`. O Código 3.2 apresenta a implementação completa.

```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "esp_log.h"
5
6  void vMonitorTask(void *pvParameters);
7
8  void app_main(void)
9  {
10     xTaskCreate(vMonitorTask, "MonitorTask", 10000, NULL, 5, NULL);
```

```
11 }
12
13 void vMonitorTask(void *pvParameters)
14 {
15     char buffer[1024];
16     for(;;)
17     {
18         TaskStatus_t *pxTaskStatusArray;
19         UBaseType_t uxArraySize, x;
20         BaseType_t xCoreID;
21         unsigned long ulTotalRunTime, ulStatsAsPercentage;
22         buffer[0] = '\0';
23
24         uxArraySize = uxTaskGetNumberOfTasks();
25         pxTaskStatusArray = pvPortMalloc(uxArraySize * sizeof(
               TaskStatus_t));
26
27         if (pxTaskStatusArray != NULL)
28         {
29             uxArraySize = uxTaskGetSystemState(
30                 pxTaskStatusArray, uxArraySize, &ulTotalRunTime);
31
32             ulTotalRunTime /= 100UL;
33
34             if (ulTotalRunTime > 0)
35             {
36                 for (x = 0; x < uxArraySize; x++)
37                 {
38                     ulStatsAsPercentage = pxTaskStatusArray[x].
                           ulRunTimeCounter / ulTotalRunTime;
39                     xCoreID = xTaskGetCoreID(pxTaskStatusArray[x].
                           xHandle);
40
41                     char *state;
42                     switch (pxTaskStatusArray[x].eCurrentState)
43                     {
44                         case eRunning:   state = "Running";   break;
45                         case eReady:     state = "Ready";     break;
46                         case eBlocked:   state = "Blocked";   break;
47                         case eSuspended: state = "Suspended"; break;
48                         default:         state = "Unknown";   break;
49                     }
50
51                     char line[128];
52                     sprintf(line,
53                         "Nome: %s | Estado: %s | Prioridade: %u | Stack:
                               %lu | Core: %d | CPU: %lu%%\n",
```

```
                        pxTaskStatusArray[x].pcTaskName,
                        state,
                        pxTaskStatusArray[x].uxCurrentPriority,
                        pxTaskStatusArray[x].usStackHighWaterMark,
                        xCoreID,
                        ulStatsAsPercentage);
                    strcat(buffer, line);
                }
            }
            vPortFree(pxTaskStatusArray);
        }

        int32_t size_memory = xPortGetFreeHeapSize();
        ESP_LOGI("MONITOR", "Memoria Livre: %ld bytes", (long int)
            size_memory);
        ESP_LOGI("MONITOR", "========= TASK MONITOR =========");
        ESP_LOGI("MONITOR", "\n%s", buffer);
        vTaskDelay(pdMS_TO_TICKS(5000));
    }

    vTaskDelete(NULL);
}
```

Listing 3.2 – Implementação da Atividade 1 — Monitor de tarefas

3.2 ATIVIDADE 2 — MEDIÇÃO DO OVERHEAD DO KERNEL

3.2.1 Descrição da Atividade

A segunda atividade teve como objetivo determinar, de forma percentual, o tempo que o *kernel* do FreeRTOS utiliza para gerenciar a troca de contexto entre tarefas no ESP32. Para isso, foram criadas duas tarefas e o intervalo de tempo exclusivo do *kernel* durante cada troca foi medido e acumulado ao longo de um período de 10 segundos, calculando-se ao final a média e o percentual de *overhead*.

3.2.2 Fundamentação Teórica

3.2.2.1 Troca de Contexto no FreeRTOS

A troca de contexto (*context switch*) é o mecanismo pelo qual o escalonador do FreeRTOS interrompe a execução de uma tarefa, salva seu estado (registradores, ponteiro de pilha e contador de programa) e restaura o estado de outra tarefa para que ela possa continuar sua execução. Esse processo é disparado pelo *tick* do sistema, gerado por uma interrupção periódica de *hardware*.

O intervalo entre o momento em que uma tarefa cede o processador e o momento em que a próxima tarefa começa a executar corresponde ao tempo exclusivo do *kernel*, denominado *overhead* de troca de contexto. Embora esse valor seja tipicamente pequeno, seu conhecimento é fundamental para validar o determinismo do sistema e dimensionar corretamente aplicações com requisitos temporais rígidos.

3.2.2.2 A Função `esp_timer_get_time()`

Para medir intervalos de tempo com precisão em microsegundos, foi utilizada a função `esp_timer_get_time()`, disponível na biblioteca `esp_timer.h` do ESP-IDF (Espressif Systems, 2025). Essa função retorna o número de microsegundos decorridos desde a inicialização do *esp_timer*, o que normalmente ocorre logo antes da chamada da função `app_main()`.

Sua assinatura é:

```
int64_t esp_timer_get_time(void);
```

Listing 3.3 – Assinatura da função esp_timer_get_time()

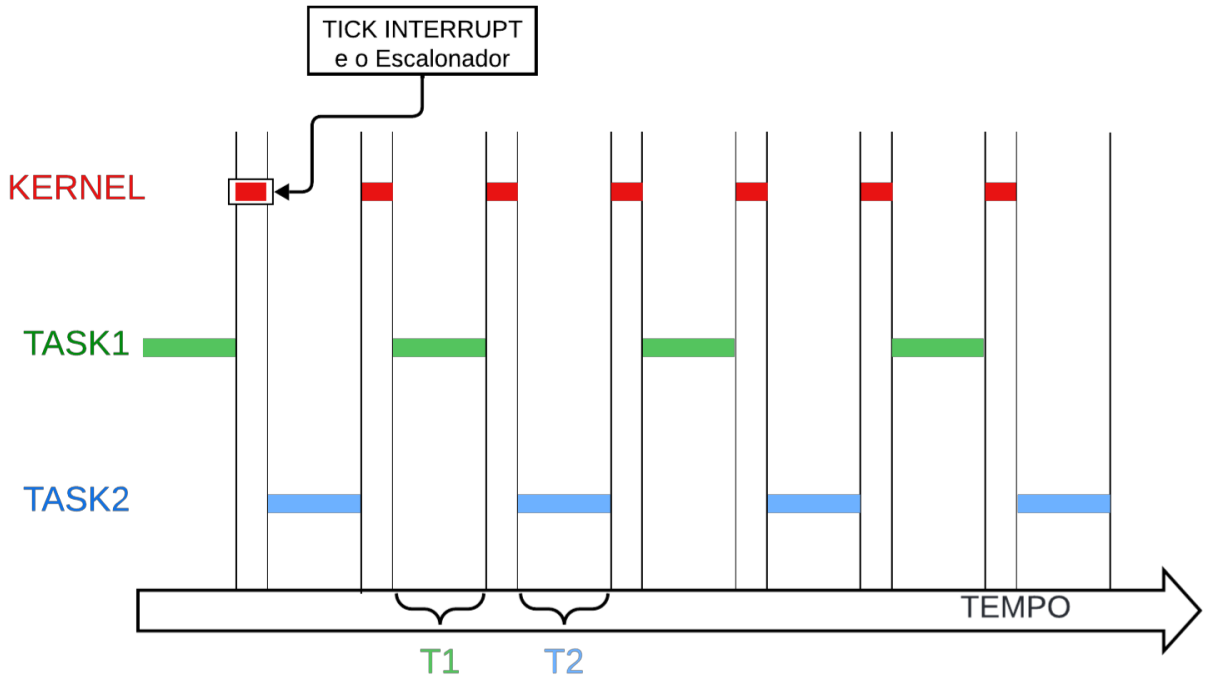
O retorno é um inteiro de 64 bits com sinal, o que garante a ausência de *overflow* em aplicações de longa duração. Diferentemente da função `gettimeofday()`, os valores retornados por `esp_timer_get_time()` partem de zero após o chip acordar de um *deep sleep*. Isso torna essa função especialmente adequada para medições de intervalos relativos de tempo, como a troca de contexto entre tarefas.

3.2.2.3 Metodologia de Medição

A estratégia adotada para isolar o tempo do *kernel* está ilustrada na Figura 4 e segue o seguinte raciocínio:

1. A **Task 1** registra seu *timestamp* de saída t_1 imediatamente antes de chamar `vTaskDelay()`;
2. O *kernel* assume o controle para realizar a troca de contexto;
3. A **Task 2** registra seu *timestamp* de entrada t_2 imediatamente ao iniciar sua execução;
4. A diferença $\Delta t = t_2 - t_1$ representa o tempo total gasto pelo *kernel* na troca de contexto.

Figura 4 – Diagrama temporal da medição de troca de contexto



Fonte: Elaborado pelo autor

Para obter maior precisão estatística, esse processo foi repetido ao longo de 1 segundo (`DURACAO = 1.000.000 µs`), acumulando-se N medições. O *overhead* médio e percentual foram então calculados conforme as Equações 3.2 e 3.3:

$$\bar{t}_{ctx} = \frac{1}{N} \sum_{i=1}^{N} \Delta t_i \tag{3.2}$$

$$\text{Overhead } (\%) = \frac{\bar{t}_{ctx}}{T_{total}} \times 100 \tag{3.3}$$

onde T_{total} corresponde à duração total do experimento em microsegundos.

3.2.3 Implementação

As duas tarefas foram fixadas no núcleo 0 via `xTaskCreatePinnedToCore()`, garantindo que a troca de contexto ocorra sempre dentro do mesmo núcleo e eliminando variáveis relacionadas à comunicação entre núcleos do ESP32 dual-core. Ambas foram criadas com a mesma prioridade (2), forçando o escalonador a alterná-las de forma regular. O Código 3.4 apresenta a implementação completa.

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
```

```
4  #include "esp_log.h"
5  #include "esp_timer.h"
6
7  #define DURACAO 1000000 // 10 segundos em microsegundos
8
9  void task1(void *parameters);
10 void task2(void *parameters);
11
12 volatile int64_t time_task1  = 0;
13 volatile int64_t time_task2  = 0;
14 volatile int64_t time_init   = 0;
15 volatile int64_t total_time  = 0;
16 volatile int     count       = 0;
17 double           media, percentual;
18
19 void app_main(void)
20 {
21     int64_t heap_size = xPortGetFreeHeapSize();
22     time_init = esp_timer_get_time();
23     ESP_LOGI("Main", "Free heap size: %lld bytes", heap_size);
24     xTaskCreatePinnedToCore(task1, "Task 1", 2048, NULL, 2, NULL, 0);
25     xTaskCreatePinnedToCore(task2, "Task 2", 2048, NULL, 2, NULL, 0);
26 }
27
28 void task1(void *parameters)
29 {
30     for (;;)
31     {
32         if ((esp_timer_get_time() - time_init) > DURACAO)
33             break;
34
35         time_task1 = esp_timer_get_time();
36         ESP_LOGI("Task 1", "Running at: %lld us", time_task1);
37         vTaskDelay(100 / portTICK_PERIOD_MS);
38     }
39     vTaskDelete(NULL);
40 }
41
42 void task2(void *parameters)
43 {
44     for (;;)
45     {
46         if ((esp_timer_get_time() - time_init) > DURACAO)
47             break;
48
49         time_task2 = esp_timer_get_time();
50         int64_t time_difference = time_task2 - time_task1;
```

```
        total_time += time_difference;
        count++;
        ESP_LOGI("Task 2", "Running at: %lld us", time_task2);
        vTaskDelay(100 / portTICK_PERIOD_MS);
    }

    media      = (double) total_time / count;
    percentual = (media / DURACAO) * 100.0;

    ESP_LOGI("Task 2", "Trocas realizadas : %d",     count);
    ESP_LOGI("Task 2", "Tempo medio        : %.2f us", media);
    ESP_LOGI("Task 2", "Overhead do kernel : %.4f %%", percentual);
    vTaskDelete(NULL);
}
```

Listing 3.4 – Implementação da Atividade 2 — Medição do overhead do kernel

3.3 ATIVIDADE 3 — PROTEÇÃO DE RECURSO COMPARTILHADO COM SEMÁFOROS

3.3.1 Descrição da Atividade

A terceira atividade consistiu em implementar um sistema produtor-consumidor com proteção de recurso compartilhado utilizando semáforos. O sistema é composto por cinco tarefas produtoras(*Temperatura*, *Umidade*, *Velocidade*, *Peso* e *Distancia*) e duas tarefas leitoras (*TaskLeitura1* e *TaskLeitura2*) que compartilham um *buffer* global de dez posições. O acesso concorrente ao recurso foi controlado por semáforos, prevenindo condições de corrida e garantindo a integridade dos dados.

3.3.2 Fundamentação Teórica

3.3.2.1 O Problema do Produtor-Consumidor

O problema do produtor-consumidor é um clássico da programação concorrente. Ele modela situações em que um ou mais produtores geram dados e os depositam em um *buffer* compartilhado, enquanto um ou mais consumidores retiram e processam esses dados. Os principais desafios são:

- **Exclusão mútua:** garantir que apenas uma tarefa acesse o *buffer* por vez, evitando corrupção de dados;
- **Sincronização:** garantir que um consumidor não tente ler de um *buffer* vazio, nem um produtor escreva em um *buffer* cheio.

3.3.2.2 Semáforo Mutex

Um *mutex* (*Mutual Exclusion Semaphore*) é um semáforo binário especial utilizado para garantir exclusão mútua no acesso a recursos compartilhados (FreeRTOS, 2024b). Quando uma tarefa deseja acessar um recurso protegido, ela deve primeiro obter o *mutex* via `xSemaphoreTake()`. Ao concluir o acesso, ela o libera com `xSemaphoreGive()`, permitindo que outra tarefa o adquira.

Uma característica importante do *mutex* no FreeRTOS é o suporte à **herança de prioridade**: se uma tarefa de alta prioridade aguarda um *mutex* mantido por uma tarefa de baixa prioridade, a prioridade desta última é elevada temporariamente para que ela conclua sua seção crítica rapidamente, mitigando o problema de inversão de prioridade.

Sua criação é feita por:

```
SemaphoreHandle_t mutex = xSemaphoreCreateMutex();
```

Listing 3.5 – Criação de semáforo mutex

3.3.2.3 Semáforos Contadores

Os semáforos contadores são criados com a função `xSemaphoreCreateCounting()`, que recebe dois parâmetros: o valor máximo que o contador pode atingir e o valor inicial do contador. Eles são ideais para rastrear a disponibilidade de recursos em quantidade, como as posições de um *buffer*. Nesta atividade foram utilizados dois semáforos contadores:

- `posicoesLivres`: inicializado com valor igual ao tamanho do *buffer* (10), representa o número de posições disponíveis para escrita. Um produtor deve decrementá-lo antes de escrever e um consumidor o incrementa após ler;
- `posicoesOcupadas`: inicializado em zero, representa o número de posições com dados disponíveis para leitura. Um produtor o incrementa após escrever e um consumidor deve decrementá-lo antes de ler.

```
SemaphoreHandle_t posicoesLivres   =
    xSemaphoreCreateCounting(BUFFER_SIZE, BUFFER_SIZE);
SemaphoreHandle_t posicoesOcupadas =
    xSemaphoreCreateCounting(BUFFER_SIZE, 0);
```

Listing 3.6 – Criação dos semáforos contadores

3.3.2.4 Uso do Parâmetro `void *parameters`

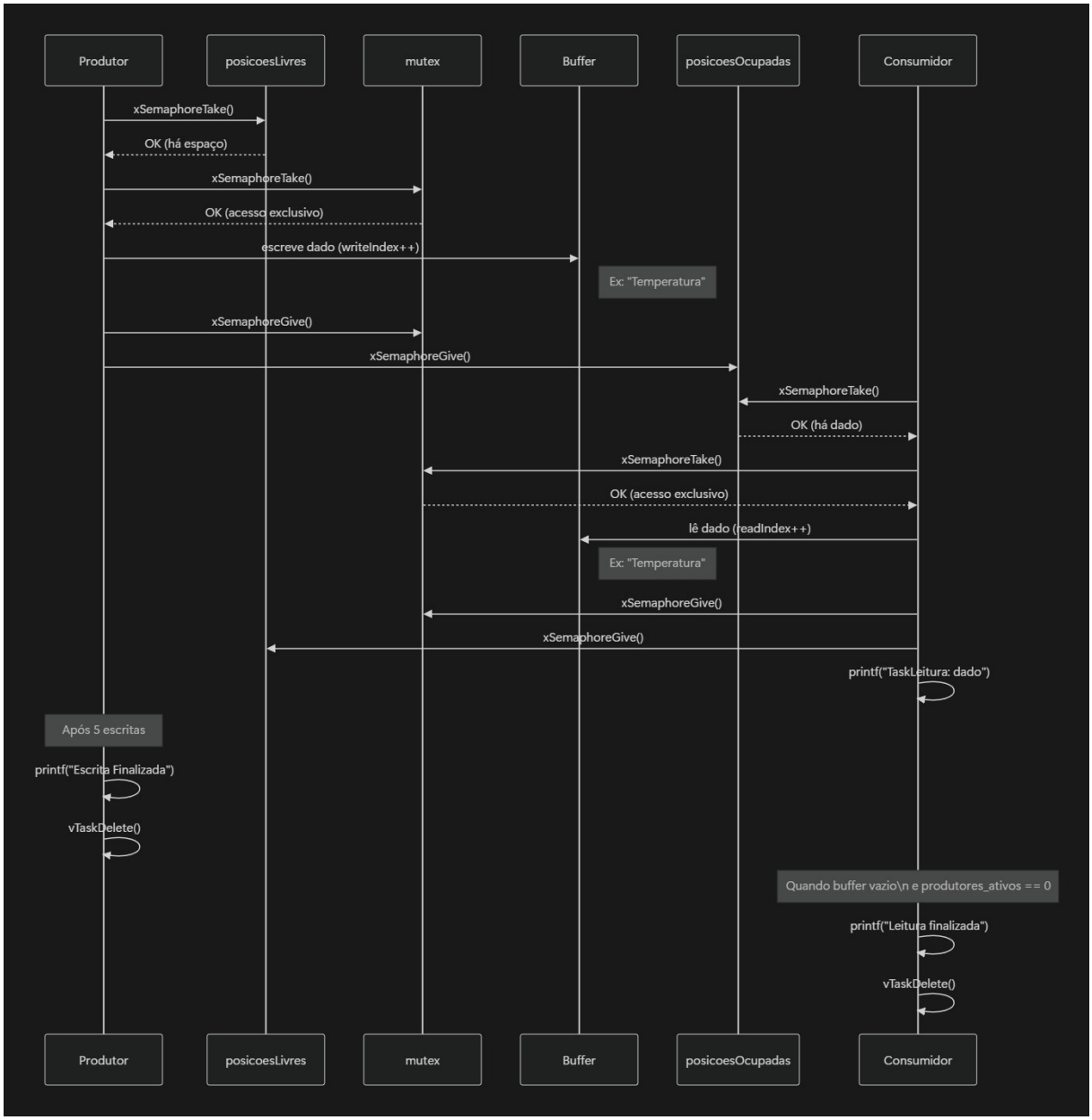
Uma decisão de projeto relevante nesta atividade foi a reutilização de uma única função de tarefa para todas as cinco tarefas produtoras e outra para as duas tarefas

leitoras, diferenciando-as pelo parâmetro `void *parameters` da `xTaskCreate()`. O nome de cada tarefa foi passado como parâmetro e recuperado via *cast* para `char *`, eliminando a necessidade de implementar sete funções distintas e tornando o código mais compacto e reutilizável.

3.3.3 Implementação

A solução foi estruturada em duas funções de tarefa (`vEscrita` e `TaskLeitura`) instanciadas sete vezes na `app_main()` com os respectivos nomes como parâmetro. Três semáforos controlam o acesso ao *buffer* circular: um *mutex* para exclusão mútua e dois semáforos contadores para sincronização entre produtores e consumidores, conforme ilustra a Figura 5.

Figura 5 – Diagrama de eventos do sistema produtor-consumidor — Atividade 3



Fonte: Elaborado pelo autor

O Código 3.7 apresenta a implementação completa.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdbool.h>
4 #include "freertos/FreeRTOS.h"
5 #include "freertos/task.h"
6 #include "freertos/semphr.h"
7 #include "esp_log.h"
8
9 #define BUFFER_SIZE 10
```

```
10
11 void vEscrita(void *parameters);
12 void TaskLeitura(void *parameters);
13
14 /* Recurso compartilhado */
15 char buffer[BUFFER_SIZE][20];
16 int writeIndex    = 0;
17 int readIndex     = 0;
18 int espacos_ativos = 5;
19
20 /* Semaforos */
21 SemaphoreHandle_t mutex;
22 SemaphoreHandle_t posicoesLivres;
23 SemaphoreHandle_t posicoesOcupadas;
24
25 void app_main(void)
26 {
27     mutex            = xSemaphoreCreateMutex();
28     posicoesLivres   = xSemaphoreCreateCounting(BUFFER_SIZE, BUFFER_SIZE
       );
29     posicoesOcupadas = xSemaphoreCreateCounting(BUFFER_SIZE, 0);
30
31     /* Tarefas produtoras */
32     xTaskCreate(vEscrita, "Temperatura", 2000, "Temperatura", 2, NULL);
33     xTaskCreate(vEscrita, "Umidade",     2000, "Umidade",     2, NULL);
34     xTaskCreate(vEscrita, "Velocidade",  2000, "Velocidade",  2, NULL);
35     xTaskCreate(vEscrita, "Peso",        2000, "Peso",        2, NULL);
36     xTaskCreate(vEscrita, "Distancia",   2000, "Distancia",   2, NULL);
37
38     /* Tarefas leitoras */
39     xTaskCreate(TaskLeitura, "TaskLeitura1", 2000, "TaskLeitura1", 2,
       NULL);
40     xTaskCreate(TaskLeitura, "TaskLeitura2", 2000, "TaskLeitura2", 2,
       NULL);
41 }
42
43 void vEscrita(void *parameters)
44 {
45     char *nome    = (char *) parameters;
46     int contador  = 0;
47
48     for (;;) {
49         if (contador >= 5) break;
50
51         xSemaphoreTake(posicoesLivres, portMAX_DELAY);
52         xSemaphoreTake(mutex, portMAX_DELAY);
53
```

```
54          strcpy(buffer[writeIndex], nome);
55          writeIndex = (writeIndex + 1) % BUFFER_SIZE;
56          contador++;
57
58          xSemaphoreGive(mutex);
59          xSemaphoreGive(posicoesOcupadas);
60
61          vTaskDelay(500 / portTICK_PERIOD_MS);
62      }
63
64      xSemaphoreTake(mutex, portMAX_DELAY);
65      espacos_ativos--;
66      xSemaphoreGive(mutex);
67
68      printf("%s: Escrita Finalizada.\n", nome);
69      vTaskDelete(NULL);
70  }
71
72  void TaskLeitura(void *param)
73  {
74      char *nome = (char *) param;
75      char dado[20];
76
77      for (;;) {
78          if (xSemaphoreTake(posicoesOcupadas, pdMS_TO_TICKS(1000)) ==
               pdTRUE)
79          {
80              xSemaphoreTake(mutex, portMAX_DELAY);
81              strcpy(dado, buffer[readIndex]);
82              readIndex = (readIndex + 1) % BUFFER_SIZE;
83              xSemaphoreGive(mutex);
84
85              xSemaphoreGive(posicoesLivres);
86              printf("%s: %s\n", nome, dado);
87          }
88          else
89          {
90              xSemaphoreTake(mutex, portMAX_DELAY);
91              int acabou = (espacos_ativos == 0);
92              xSemaphoreGive(mutex);
93
94              if (acabou) break;
95          }
96      }
97
98      printf("%s: Leitura finalizada!\n", nome);
99      vTaskDelete(NULL);
```

```
100 }
```

Listing 3.7 – Implementação da Atividade 3 — Produtor-consumidor com semáforos

4 RESULTADOS

Neste capítulo são apresentados os resultados obtidos em cada atividade, especificando as principais informações adquiridas após a execução dos seus respectivos códigos.

4.1 Atividade 1

A Figura 6 apresenta a saída obtida no monitor serial após a execução da `vMonitorTask`.

Figura 6 – Saída do monitor serial

```
I (50807) MONITOR: Memória Livre: 298908 bytes
I (50807) MONITOR: =============== TASK MONITOR ===============
I (50807) MONITOR:
Nome: MonitorTask | Estado: Running | Prioridade: 5 | Stack: 6988 | Core: 2147483647 | Tempo de CPU: 528264 | Uso de CPU: 1%
Nome: IDLE1 | Estado: Ready | Prioridade: 0 | Stack: 1024 | Core: 1 | Tempo de CPU: 50509969 | Uso de CPU: 99%
Nome: IDLE0 | Estado: Ready | Prioridade: 0 | Stack: 1028 | Core: 0 | Tempo de CPU: 49976542 | Uso de CPU: 98%
Nome: ipc1 | Estado: Suspended | Prioridade: 24 | Stack: 504 | Core: 1 | Tempo de CPU: 33203 | Uso de CPU: 0%
Nome: ipc0 | Estado: Suspended | Prioridade: 24 | Stack: 496 | Core: 0 | Tempo de CPU: 32453 | Uso de CPU: 0%
```

Fonte: Elaborado pelo autor

A Tabela 3 sintetiza as tarefas identificadas e suas respectivas funcionalidades, com base na saída do monitor e na documentação do ESP-IDF (Espressif Systems, 2025).

Tabela 3 – Tarefas do sistema identificadas na Atividade 1

Nome	**Estado**	**Prio.**	**Core**	**Funcionalidade**
`MonitorTask`	Running	5	0	Tarefa criada pelo usuário para monitoramento das demais tarefas
`IDLE0`	Ready	0	0	Tarefa ociosa do núcleo 0 — executa quando nenhuma outra tarefa está pronta
`IDLE1`	Ready	0	1	Tarefa ociosa do núcleo 1 — executa quando nenhuma outra tarefa está pronta
`ipc0`	Blocked	24	0	*Inter-Processor Call* — executa chamadas entre núcleos
`ipc1`	Blocked	24	1	*Inter-Processor Call* — executa chamadas entre núcleos

Fonte: Elaborado pelo autor

4.2 Atividade 2

A Figura 7 apresenta a saída obtida no monitor serial ao final da execução do experimento.

Figura 7 – Saída do monitor serial — Atividade 2

```
I (1191) Task 1: Task 1 is running at time: 942321
I (1191) Task 2: Task 2 is running at time: 942451
I (1201) Task 2: Task 2 is running at time: 952320
I (1211) Task 2: Task 2 is running at time: 962320
I (1221) Task 2: Task 2 is running at time: 972320
I (1231) Task 2: Task 2 is running at time: 982320
I (1241) Task 2: Task 2 is running at time: 992320
I (1251) Task 2: Task 2 is running at time: 1002320
I (1261) Task 2: Task 2 is running at time: 1012320
I (1271) Task 2: Task 2 is running at time: 1022320
I (1281) Task 2: Task 2 is running at time: 1032320
I (1291) Task 1: Task 1 is running at time: 1042321
I (1291) Task 2: Task 2 is running at time: 1042453
I (1301) Task 2: Trocas: 101
I (1301) Task 2: Tempo medio 44459 us
I (1301) Task 2: Overhead do kernel: 4.4459 %
```

Fonte: Elaborado pelo autor

A Tabela 4 sintetiza os resultados obtidos.

Tabela 4 – Resultados da medição de overhead do kernel — Atividade 2

Métrica	**Valor**
Duração do experimento	1.000.000 µs (1 s)
Número de trocas de contexto (N)	101
Tempo médio de troca ($\bar{t}_{ctx}$)	44459 µs
Overhead percentual do kernel	4,4459 %

Fonte: Elaborado pelo autor

Os resultados evidenciam que o *overhead* do *kernel* do FreeRTOS representa uma fração muito pequena do tempo total de execução, o que confirma o caráter determinístico e eficiente do escalonador para aplicações embarcadas com requisitos de tempo real. Esse baixo *overhead* é consequência direta das operações internas de tempo constante adotadas pelo FreeRTOS no processo de troca de contexto (FreeRTOS, 2024a).

4.3 Atividade 3

A Figura 8 apresenta a saída obtida no monitor serial durante a execução do sistema.

Figura 8 – Saída do monitor serial — Atividade 3

```
TaskLeitura1: : Temperatura
TaskLeitura2:: Velocidade
TaskLeitura1: : Umidade
TaskLeitura2:: Distancia
TaskLeitura1: : Peso
Temperatura: Escrita Finalizada.
Velocidade: Escrita Finalizada.
Umidade: Escrita Finalizada.
Distancia: Escrita Finalizada.
Peso: Escrita Finalizada.
TaskLeitura2:: Leitura finalizada!
TaskLeitura1: : Leitura finalizada!
```

Fonte: Elaborado pelo autor

A saída evidencia o correto funcionamento do mecanismo de sincronização: as tarefas produtoras escrevem no *buffer* de forma intercalada, sem sobrescrever dados não lidos, e as tarefas leitoras consomem cada posição exatamente uma vez, encerrando-se ordenadamente ao detectar que todas as escritas foram finalizadas e o *buffer* está vazio.

É importante destacar que, por se tratarem de tarefas de mesma prioridade operando sobre um *buffer* compartilhado, a ausência dos semáforos resultaria em condição de corrida (*race condition*): múltiplas tarefas poderiam ler ou escrever simultaneamente na mesma posição do *buffer*, corrompendo os dados. A combinação do *mutex* com os semáforos contadores resolve ambos os problemas de exclusão mútua e sincronização (FreeRTOS, 2024a).

5 CONCLUSÃO

O presente trabalho explorou os fundamentos do desenvolvimento de aplicações com FreeRTOS no ESP32, utilizando o ESP-IDF como *framework* de suporte. As três atividades propostas permitiram uma progressão natural do aprendizado, da observação do comportamento interno do sistema, passando pela medição de desempenho do *kernel*, até a implementação de mecanismos de sincronização entre tarefas concorrentes.

Na Atividade 1, a criação da tarefa de monitoramento evidenciou que o ESP-IDF, ao ser inicializado, já instancia automaticamente um conjunto de tarefas internas essenciais para o funcionamento do sistema, como as tarefas ociosas `IDLE0` e `IDLE1` e as tarefas de comunicação entre processadores `ipc0` e `ipc1`, ambas com prioridade máxima de 24. Além das informações básicas exigidas pela atividade, o código foi incrementado com o cálculo do percentual de uso de CPU por tarefa e com o monitoramento da memória *heap* disponível via `xPortGetFreeHeapSize()`, demonstrando como a API do FreeRTOS oferece instrumentos eficazes para introspecção do sistema em tempo de execução.

Na Atividade 2, a medição do *overhead* do *kernel* durante a troca de contexto entre duas tarefas fixadas no núcleo 0 revelou um tempo médio de troca de 44.459 μs, correspondente a um *overhead* percentual de $4,4459\%$ ao longo de um experimento de 1 segundo com 101 trocas registradas. Esse valor, embora mensurável, reflete em grande parte o intervalo de `vTaskDelay()` de 100 ms configurado entre as execuções das tarefas, ou seja, o tempo medido engloba não apenas o custo da troca de contexto em si, mas também o período em que as tarefas permaneceram bloqueadas aguardando o *delay*. O custo exclusivo da comutação de contexto pelo *kernel* é, na prática, da ordem de poucos microsegundos, reforçando o caráter determinístico e eficiente do escalonador do FreeRTOS.

Na Atividade 3, a implementação do sistema produtor-consumidor com cinco tarefas escritoras e duas tarefas leitoras demonstrou na prática a importância dos mecanismos de sincronização em sistemas concorrentes. A combinação de um semáforo *mutex* com dois semáforos contadores garantiu a integridade dos dados e a coordenação correta entre produtores e consumidores. Uma contribuição relevante desta implementação foi a reutilização de uma única função de tarefa para múltiplas instâncias, diferenciadas pelo parâmetro `void *parameters`, o que evidencia boas práticas de projeto de *software* embarcado.

De forma geral, as atividades realizadas consolidaram o entendimento sobre os principais pilares do FreeRTOS: criação e gerenciamento de tarefas, escalonamento baseado em prioridades, medição de desempenho em tempo real e sincronização com semáforos. A integração com o ESP-IDF mostrou-se robusta e bem documentada, oferecendo APIs de

alto nível que facilitam o desenvolvimento de aplicações embarcadas confiáveis e determinísticas sobre o hardware dual-core do ESP32.

Referências

Espressif Systems. *ESP32-WROOM-32 Datasheet*. [S.l.], 2023. Disponível em: <https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf>. Acesso em: 29 mar. 2026.

Espressif Systems. *ESP-IDF Programming Guide v5.5.1*. [S.l.], 2025. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32/index.html>. Acesso em: 29 mar. 2026.

FreeRTOS. *FreeRTOS Reference Manual*. [S.l.], 2024. Disponível em: <https://www.freertos.org/Documentation/00-Overview>. Acesso em: 29 mar. 2026.

FreeRTOS. *Semaphores and Mutexes — FreeRTOS Reference Manual*. [S.l.], 2024. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/04-API-references/10-Semaphore-and-Mutexes/00-Semaphores>. Acesso em: 29 mar. 2026.

FreeRTOS. *uxTaskGetSystemState() — FreeRTOS Task Utilities*. 2024. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/04-API-references/03-Task-utilities/01-uxTaskGetSystemState>. Acesso em: 29 mar. 2026.

FreeRTOS. *xPortGetFreeHeapSize() — FreeRTOS heap memory management*. 2024. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/09-Memory-management/01-Memory-management>. Acesso em: 29 mar. 2026.

NETO, V. G. P. *ELE0629_TESEM: Repositório de atividades da disciplina de Tópicos Especiais em Sistemas Embarcados*. 2026. Disponível em: <https://github.com/Valdemar-Neto/ELE0629_TESEM>. Acesso em: 29 mar. 2026.